Problem Chosen	2022 MCM/ICM Summary Sheet	Team Control Number
C		2220954

Beat The Market:Trading Model Based on History Price

Summary

It is essential for investors to adopt an effective investment strategy to hedge risk and ensure investment returns when trading in volatile assets.Most investors are aware that the market undergoes times of strong trends.Making the wrong moves could wipe out previous gains and more. Through understanding volatility, we can create appropriate trading strategies. that help to harness profit potential. This can be done by trading volatile assets, tracking changes in volatility to aid in selection, incorporating volatility-based technical indicators or software, or by focusing on low-volatility assets.

Volatility is an important metric for all traders, including short-term day traders and swings traders, whose primary focus is on daily and weekly price movements.

Recently, the use of mathematical models to solve quantitative financial problems has become a hot spot. Mathematical models can be used for quantitative analysis of most objective problems, especially in the field of finance. At present, mathematical models have become a tool used by professionals in the financial field. Therefore, with the continuous development of science and technology economy and the continuous innovation of mathematical models, there are constantly new challenges in solving financial problems based on mathematical models.

To solve this problem, both two-stage models and reinforcement learning based models are proposed. The two-stage models consist of two parts – prediction and optimization. The former predicts the expectation price using the available data. The later does trading optimization by the prediction and make deal at a specific frequency. Operations research, economics, machine learning and neural networks are used to form the model. In our work, XGboost[1], SVR[2] and LSTM-net[3] are adopted to make prediction and Genetic Algorithm is implemented to handle trading optimization.

The reinforcement based models formulate the problem as a Markov decision process(MDP) and use a popular state-of-the-art algorithm Twin Delayed Deep Deterministic policy gradient algorithm(TD3)[4] to solve the problem well. Also, the latest technologies are well implemented in this model like deep learning, LSTM-net[3] and policy gradient.

Both of the categories of models can achieve a acceptable profit. The two-stage models earn $4475255usd$ in this problem and the reinforcement model earn $524811usd$. We have enough confident that our models have solved this problem well and are helpful to industry. We use prediction and the optimization comparison presenting evidence that our model provides the best strategy. In section 4.4, we do experiment on how do transaction costs affect the strategy and results.

Keywords: Portfolio management , LSTM, XGBoost, Reinforcement-learning.

Contents

1 Introduction **3**
1.1 Problem Background . . . 3
1.2 Literature Review . . . 3

2 Problem definition **4**
2.1 Notations . . . 4
2.2 Assumption . . . 4
2.3 Formulation . . . 4

3 Models **5**
3.1 Price Prediction . . . 5
3.1.1 SVR . . . 5
3.1.2 XGBoost . . . 6
3.1.3 LSTM . . . 6
3.2 Prediction model evaluation . . . 7
3.3 Trading optimization . . . 7
3.3.1 Genetic Algorithm . . . 7
3.4 Reinforcement learning based methods: . . . 8
3.4.1 Markov decision process: . . . 8
3.4.2 Model . . . 9
3.4.3 Algorithm . . . 10

4 Experiments **10**
4.1 Hardware and software . . . 10
4.2 Two stage model performance . . . 10
4.2.1 Prediction model performance . . . 10
4.2.2 GA algorithm performance . . . 12
4.3 End to end model performance . . . 13
4.4 Sensitiveness experiment . . . 14

5 Strengths and Weaknesses **14**
5.1 Two stage model . . . 14
5.1.1 Strengths . . . 14
5.1.2 Weaknesses . . . 14
5.2 End to end model . . . 16

Memo to Market trader **17**

References **18**

Appendix A: Algorithms **20**
Appendix Gentic Algorithm: . . . 20
Appendix Deep Reinforcement Learning algorithm: . . . 20

Appendix B: Program Codes **21**

1 Introduction

1.1 Problem Background

Multi-objective linear programming has two or more objective functions, and the objective functions and constraints are all linear functions. Algorithmic trading[5], also known as automatic trading, black-box trading or machine trading, refers to the method of issuing trading instructions by designing algorithms and using computer programs. In trading, the range that the program can decide includes the choice of trading time, the price of the transaction, and even the number of assets that need to be traded at the end.

The machine learning or deep learning that we use is to solve a big data problem. But in financial data, we may be dealing with a small sample of data. This makes it very difficult to apply machine learning to automate learning, and with deep learning it is easy to have overfitting problems. At the same time, we can view the time series of prices as a partially observable Markov decision process (POMDP) . But at no time can anyone draw the complete price time series very completely. We never know what will happen tomorrow, but we still need to make decisions about our trades. At the time we make a decision, we have very little data, and at the same time, the distribution of data is constantly changing.

Two major problems are discussed in this paper, which are:

- How to predict the future price based on history data accurately .
- How to design scientific model to decide the buy and sell.

1.2 Literature Review

For this kind of price prediction problem, regression model can be used for modeling and solving. Firstly, the model is used to fit the given historical data, and a model with a high degree of fitting is obtained.Then use the last part of the data to detect and verify the model, adjust and optimize the model. Finally, according to the given data, the model is used to predict the results. The implementation of a model combination with Genetic Algorithms (GA), Artificial Neural Networks and Hidden Markov Model (HMM) was proposed by Hassan et al. [6]; the purpose was transforming the daily stock values to independent groups of prices as inputs to HMM. The predictability of financial trend with SVM model by evaluating the weekly trend of NIKKEI 225 index was investigated by Huang et al. [7].

It can be seen from the above research background that each algorithm can effectively solve the stock prediction problem. However, it is important to note that each has specific limitations. The prediction result is not only affected by the representation method of input data, but also depends on the prediction method. Furthermore, using only salient features and using them as input data, rather than all features, can significantly improve the accuracy of the prediction model.

At the same time, we use reinforcement learning to deal with this problem. Reinforcement learning methods have b widely adopted to deal with quantitative transactions.[8][9][9]

2 Problem definition

2.1 Notations

The primary notations used in this paper are listed in Table 1.

Table 1: Notations

Symbol	Definition
H_g	the holding value of gold
H_b	the holding value of bitcoin
T_g	the trading value of gold
T_b	the trading value of bitcoin
H_d	the holding value of dollar
$\mathbf{g}$	today price of the troy ounces
$\mathbf{b}$	today price of the bitcoins
$\mathbf{g'}$	predict price of the troy ounces
$\mathbf{b'}$	predict price of the bitcoins
α_{gold}	commission of gold for each transaction
$\alpha_{bitcoin}$	commission of bitcoin for each transaction

2.2 Assumption

As works of predecessors, We can make assumptions to the execution of portfolio transactions in this problem as follow. The gold and bitcoin can only trade at most once in one turn.We estimate the value of assets now by usd and the goal is to optimize it with the constraints which limit the search space of GA algorithm.

2.3 Formulation

The problem can be defined as an optimization problem while the goal is to earn more money. Notice that gold can get deal on weekday$T$.

$$
\begin{aligned}
&\text{if gold is able to trade:}\\
&\max \frac{g'(T_g+H_g)}{g}+\frac{b'(T_b+H_b)}{b}+(H_d-T_g(1-\alpha_{gold}\text{ sign }H_g)-T_b(1-\alpha_{bitcoin}\text{ sign }H_b))\\
\text{s.t. }&-H_g\le T_g\le\frac{H_d}{(1+\alpha_{gold})}\\
&-H_b\le T_b\le\frac{H_d}{(1+\alpha_{bitcoin})}\\
&-H_b-H_g\le T_g+T_b\le H_d\\
&\alpha_{bitcoin}\le(\frac{b'}{b}-1)\text{ signed }H_b\\
&\alpha_{gold}\le(\frac{g'}{g}-1)\text{ sign}H_g\\
&\text{else:}\\
&\max g+\frac{b'(T_b+H_b)}{b}+(H_d-T_b(1-\alpha_{bitcoin}\text{ sign }H_b))\\
\text{s.t. }&-H_b\le T_b\le\frac{H_d}{(1+\alpha_{bitcoin})}\\
&\alpha_{bitcoin}\le(\frac{b'}{b}-1)\text{ sign }H_b
\end{aligned}
\tag{1}
$$

3 Models

This is a regression prediction problem. For daily price prediction of Bitcoin and gold, we used SVR, XGBoost and LSTM models respectively, and compared the prediction performance using several evaluation metrics, and finally determined the model with the best prediction performance. For trading decisions, we use genetic algorithms and reinforcement learning-based algorithms to determine the trading strategy, respectively, and choose the optimal algorithmic model based on the final return results

3.1 Price Prediction

3.1.1 SVR

The Support Vector Regression (SVR, shown in Figure)[2] model uses SVM to fit curves and perform regression analysis.SVM is a binary classification algorithm that supports both linear and nonlinear classification. It has evolved to now support multivariate classification and regression problems .For the low-dimensional feature data that is linearly inseparable by SVM, we can map it to high-dimensional, and it can be linearly separable. It can produce good prediction result if the value of important parameters can be determined properly.

3.1.2 XGBoost

XGBoost [1] makes some improvements to Boosting algorithm on the basis of GBDT model, and its internal decision tree uses regression tree.Boosting algorithm's inherent idea is to group several weak classifiers together to form a strong classifier.

The main advantages of the XGBoost model include a regular term in the target function to prevent overfitting.Its parallel optimization is on the feature granularity, which is more efficient.Supporting column sampling can not only reduce the over-fitting, but also reduce the amount of calculation.Considering that the training data is sparse, the default direction of the branch can be specified for some values.[10]

3.1.3 LSTM

LSTM (Long Short Term Neural Network)[3] is a recurrent neural network that can learn and predict time series.The LSTM architecture is a memory cell, which can maintain its state over time, and nonlinear gating units, which regulate the information flow into and out of the cell.

LSTMs are widely used for sequence prediction problems and have proven to be extremely effective. The reason they work so well is that LSTM can store past important information and forget the information that is not.

LSTM has three gates:

- The input gate: The input gate adds information to the cell state,
- The forget gate: It removes the information that is no longer required by the model,
- The output gate: Output Gate at LSTM selects the information to be shown as output.

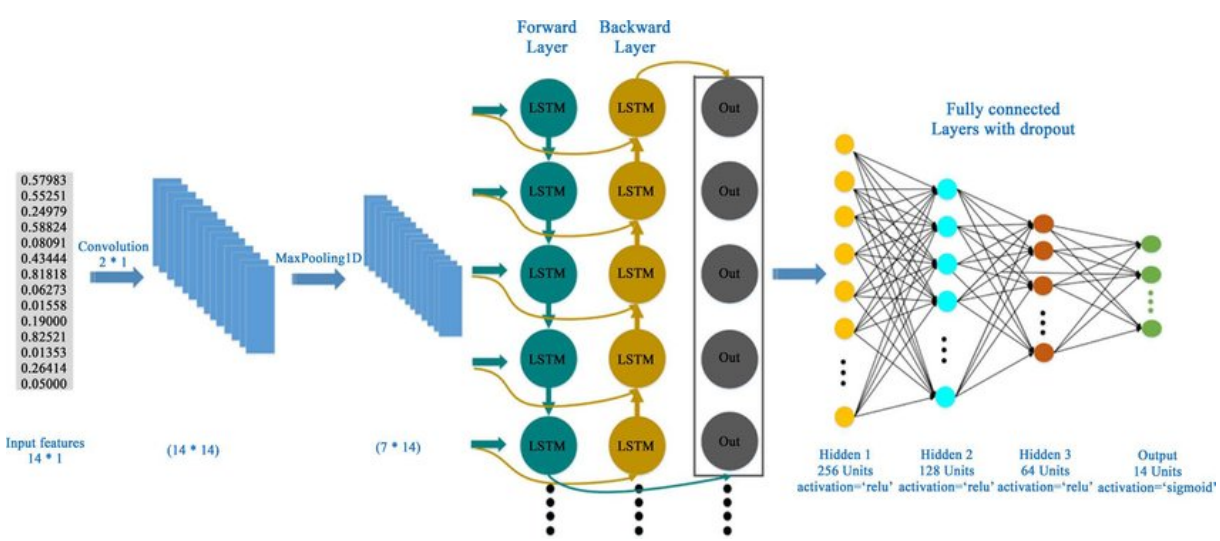

Figure 1: LSTM

3.2 Prediction model evaluation

Prediction accuracy indicates how well a prediction model predicts the selected variables. Different accuracy measures are used to validate the fitness of a model for a particular data set. Typically, there are several accuracy measures such as mean error (ME), mean absolute error (MAE), mean absolute percentage error (MAPE), mean squared error(MSE) and mean square root error (RMSE). The following metrics are calculated and compared:

- MSE = $\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$
- RMSE = $\sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$
- MAE = $\frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$
- R2_Score = $1 - \frac{\sum_{i=1}^{n}(\hat{y}_i - y_i)^2}{\sum_{i=1}^{n}(\bar{y}_i - y_i)^2}$

3.3 Trading optimization

3.3.1 Genetic Algorithm

Genetic Algorithm (GA) is a computational model that simulates the natural selection of Darwin's theory of biological evolution and the biological evolution process of the genetic mechanism. It is a method to search for optimal solutions by simulating the natural evolution process.In our model,we use GA to optimize the target function we make in Section 2.3 and we combine the long run price and short run price to estimate the value.

Initialization The population size (denoted as u) depends on the nature of the problem, but typically contains several hundreds or thousands of possible solutions. Often, the initial population is generated randomly, allowing the entire range of possible solutions (the search space). Occasionally, the solutions may be 'seeded' in areas where optimal solutions are likely to be found.

Selection Normally, selection in genetic algorithms is a probabilistic operator which uses the relative fitness $p_s(\vec{x}_i) = f(\vec{x}_i) / \sum_{j=1}^{u} f(\vec{x}_j)$ to serve as selection probabilities (u denotes the population size). This selection operator is called proportional selection. It is commonly used; as intuitively the more fit a solution is the more we want it to affect the next generations.
If the problem under consideration is a minimization one, or if the fitness function can take negative values, then $f(\vec{x}_i)$ has to be linearly transformed before calculating selection probabilities. This technique known as Linear Dynamic Scaling is commonly used in genetic algorithms .

Crossover The crossover operator allows the exchange of information between different individuals. The original one-point crossover works on two parent individuals by choosing a crossover point $\chi \in \{1, \ldots, n-1\}$ at random, and exchanging all bits after the χth one between both individuals. Therefore, two new off-springs will be formed.

The crossover rate p_c determines the probability to undergo crossover , this allows the two parents to be present in the next generation; as crossover is not mandatory. Moreover, the value of p_c indicates the ratio of new off-springs in the next generation.
The crossover operator can be extended to a generalized multi-point crossover or even to uniform crossover, where it is randomly decided for each bit whether to exchange it or not. The strong mixing effect introduced by uniform crossover is sometimes helpful to overcome local optima.

Mutation New information is introduced into the population by means of mutation, which works by inverting bits with a small probability p_m Though mutation is often interpreted as a rather unimportant operator in genetic algorithms , recent theoretical work gives strong evidence for an appropriate choice of a mutation rate $p_m = 1/n$ on many problems .

3.4 Reinforcement learning based methods:

Instead of using two stage methods to solve the problem, one-stage methods are also outstanding. To develop a model that gives the best daily trading strategy based only on price data up to that day, the problem can be defined as a Markov problem and reinforcement learning[11] based models are naturally implemented.

In this section, a model which is able to decide the ratio of three currencies is implemented. Instead of using two model to solve the problem, one-stage method realize an end-to-end network and simplifies the processAlso, designing a one-stage model may use less expert experience.

Nowadays, lots of researchers have adopted RL to financial quantification. The application of reinforcement learning in financial quantification especially in portfolio management are hot pots. In this section, we model the problem as a continuous control problem, and use the most popular continuous control algorithms DDPG[12] to deal with this problem. TD3[4] algorithm is an upgraded version of DDPG which has achieved great performance. These two algorithms have achieved perfect applications in combinatorial optimization[13], robotics[14] and other fields.

Then, we will first formulate our Markov decision process and then introduce our models.

3.4.1 Markov decision process:

As shown in figure above, the daily price fit a **Markov random process**.

state: The state of our MDP is consist of the price of last k days, where the k is set to 25 in my model. Although in fact, the state of Markov process should be related to a series of factors such as market public opinion and politics, we still believe that it is a Markov process rather than a partially observable Markov process(POMDP) to easily train models.

action: The action of our MDP is a vector $a = (a_{gold}, a_{bitcoin})$, representing how optimistic the agent is about the stock price. Both numbers are continuous variable from -1 to 1. If $a_gold, a_bitcoin \in (-0.2, 0.2)$ we regard that the agent wants to maintain the status and do nothing. And agent will buy more stock which $a \geq 0.2$ and sell the stock which $a \leq 0.2$.

reward: The reward is the real earned assets. We use both 1-day reward and 5-day reward to make our agent get long-term optimal.

$$\begin{aligned} r(s_t, a_t) =& [(g_{s_{t+1}} - g_{s_t} * \alpha_{gold}) + 0.2 * (g_{s_{t+4}} - g_{s_t} * \alpha_{gold})] * a_{gold} \\ &+ [(b_{s_{t+1}} - b_{s_t} * \alpha_{bitcoin}) + 0.2 * (b_{s_{t+4}} - b_{s_t} * \alpha_{bitcoin})] * a_{bitcoin} \end{aligned} \tag{2}$$

policy: We use DDPG algorithm[12] to solve this problem so the policy is Deterministic strategy.

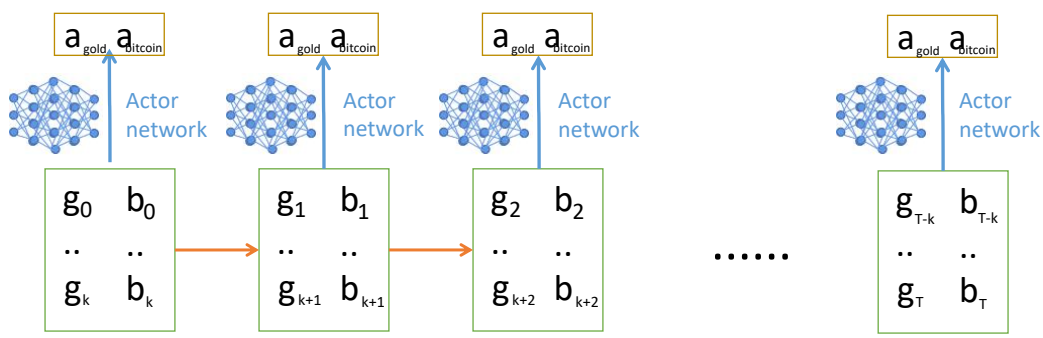

Figure 2: the MDP

3.4.2 Model

Deep deterministic policy gradient(DDPG)[12] algorithm is implemented to deal with this problem. For our problem can be modeled as MDP, the value function(V-function) and the action function(Q-function) conforms to the Bellman equation.

The update methods of networks and loss functions are as follows. Where critic networks Q_θ, and actor network π_ϕ have random parameters θ, ϕ. Critic network will update to decrease the MSE loss between reward and its output.

$$
\begin{aligned}
R_t &= \sum_{i=t}^{T} \gamma^{i-t} r(s_i, a_i) \\
Q^{\pi}(s,a) &= \mathbb{E}_{s_i \sim p_\pi, a_i \sim \pi}[R_t | s, a] \\
J(\phi) &= \mathbb{E}_{\mathbf{s} \sim p_\pi}[R_0] \\
\nabla_\phi J(\phi) &= \mathbb{E}_{\mathbf{s} \sim p_\pi}[\nabla_a Q^{\pi}(s,a)|_{a=\pi_\phi(s)} \nabla_\phi \pi_\phi(s)]
\end{aligned}
\tag{3}
$$

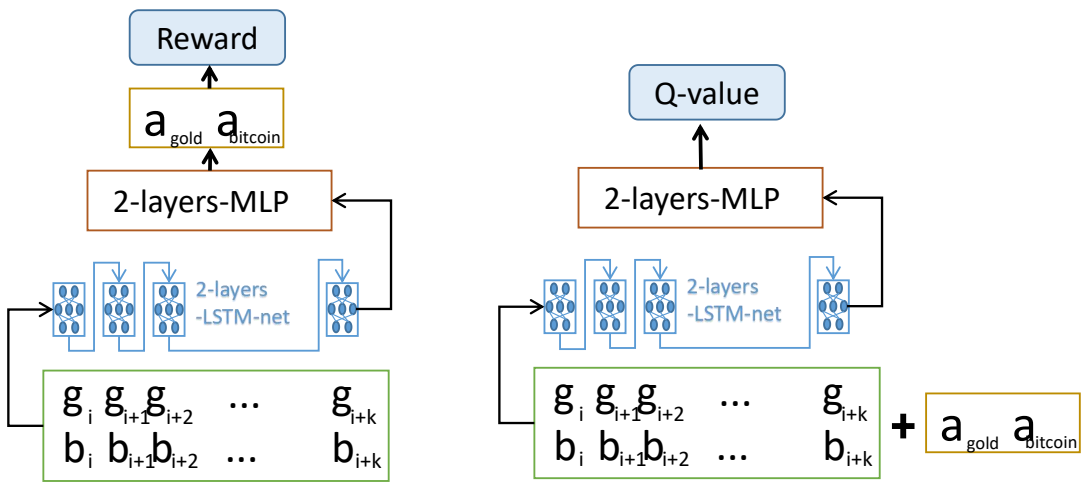

Figure 3: the network structure, the left one is Actor network and the right one is Critic network

3.4.3 Algorithm

After knowing the loss function and MDP model, the training method of RL model is easy to be clear, so we leave the details of both algorithm in the appendix A.

4 Experiments

4.1 Hardware and software

The models are trained on Intel(R) Core(TM) i7-10750H CPU @ 2.60GHz 2.59 GHz and nvidia 2080-Ti and the operating system is Win10 .

4.2 Two stage model performance

4.2.1 Prediction model performance

The daily price forecast line charts for gold and bitcoin from the above model are as follows, Which are shown in figure [4,5]. The results of the comparison of the predictive performance of the above models using evaluation metrics are as follows, which are shown in tables[2,3]

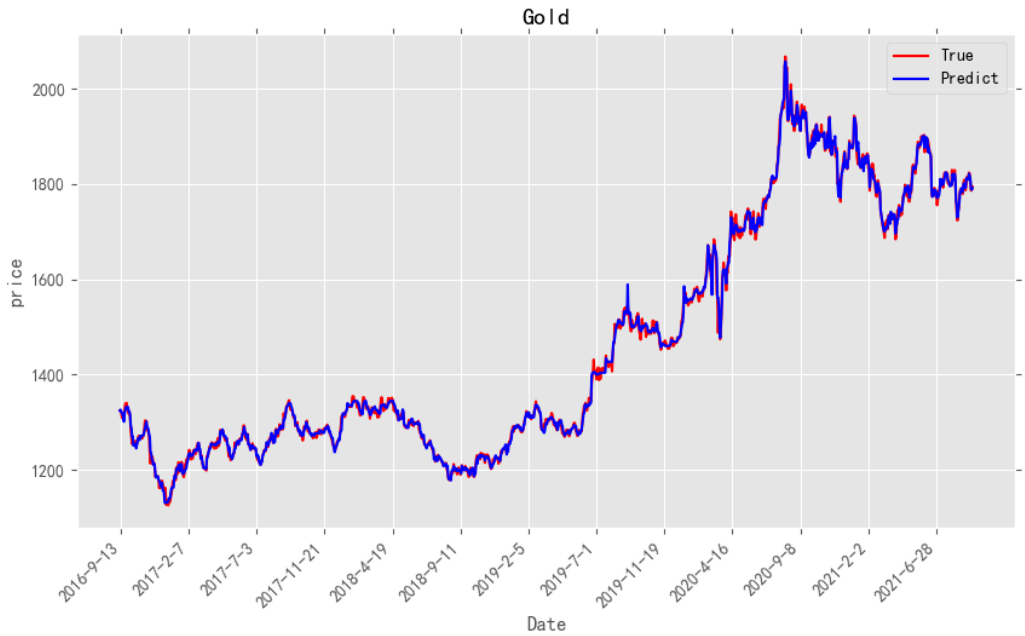

Figure 4: gold price prediction by XGBoost

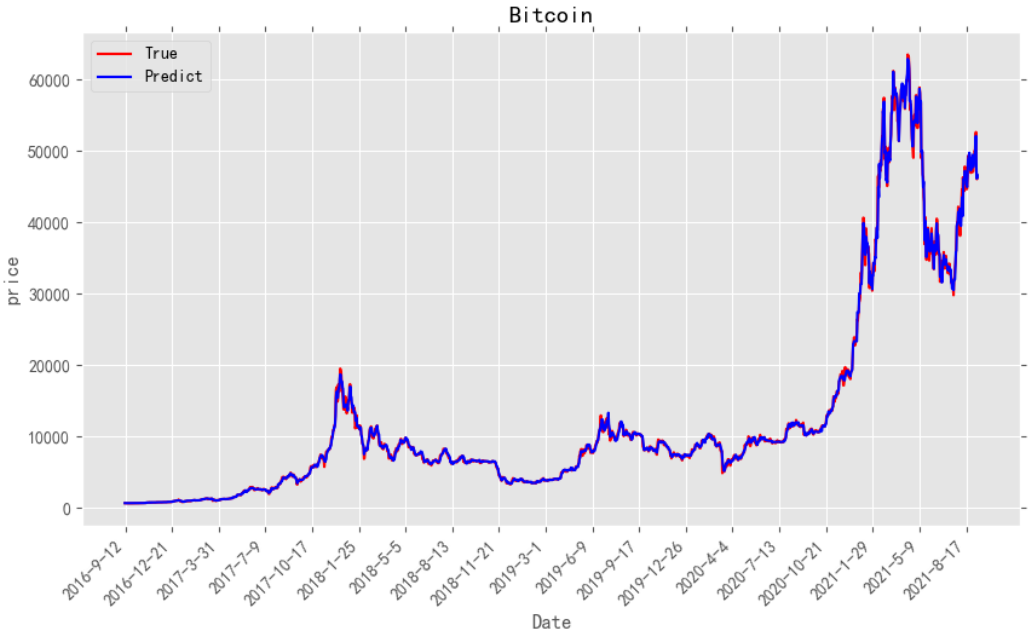

Figure 5: BCHAIN-MKPRU price prediction by XGBoost

Table 2: Prediction of Bitcoin

Method	MSE	RMSE	MAE	R2
SVR	630496	794.04	392.01	0.9968
XGBoost	178650	422.67	210.98	0.9991
LSTM	1679682	1296.03	1110.49	0.9915

Table 3: Prediction of Gold

Method	MSE	RMSE	MAE	R2
SVR	226	15.04	10.31	0.9964
XGBoost	77	8.80	5.82	0.9988
LSTM	285	16.89	12.89	0.9954

4.2.2 GA algorithm performance

Using the XGBoost model to predict the price in long run and short run,we use GA model to decide the buy and sell.The model gain 4475255usd at last.We can observe that the line is like a exponential function.

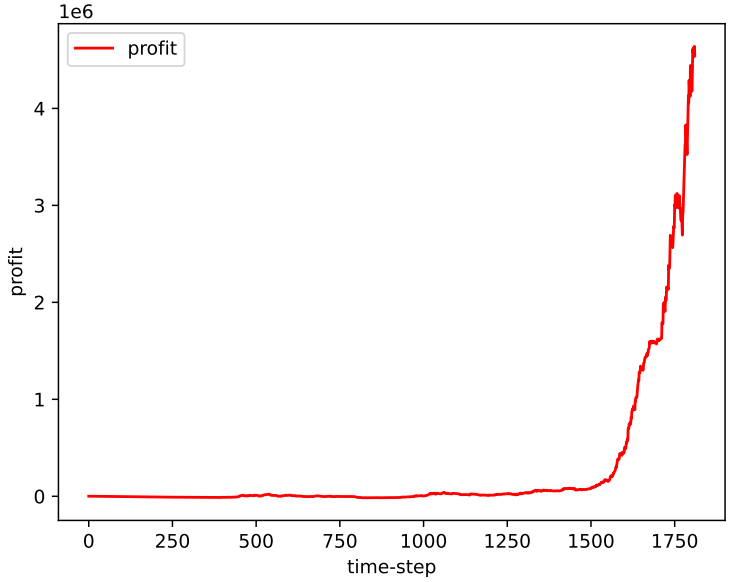

Figure 6: The profit by GA algorithm

4.3 End to end model performance

The reward function of training and the total profit are as follow. We can observe that the network chose bitcoin well when the price of bitcoin soared, and successfully avoided risk when the price of bitcoin fellThe final profit of the method is 524810.87555usd.

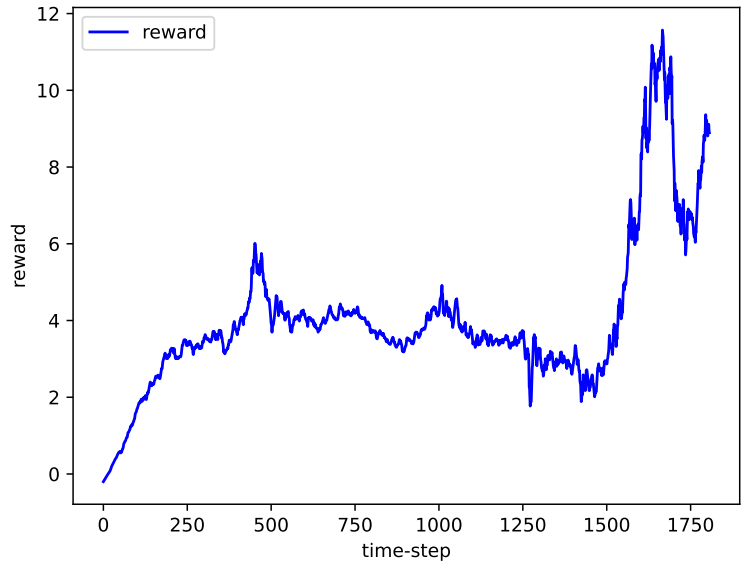

Figure 7: The reward while training

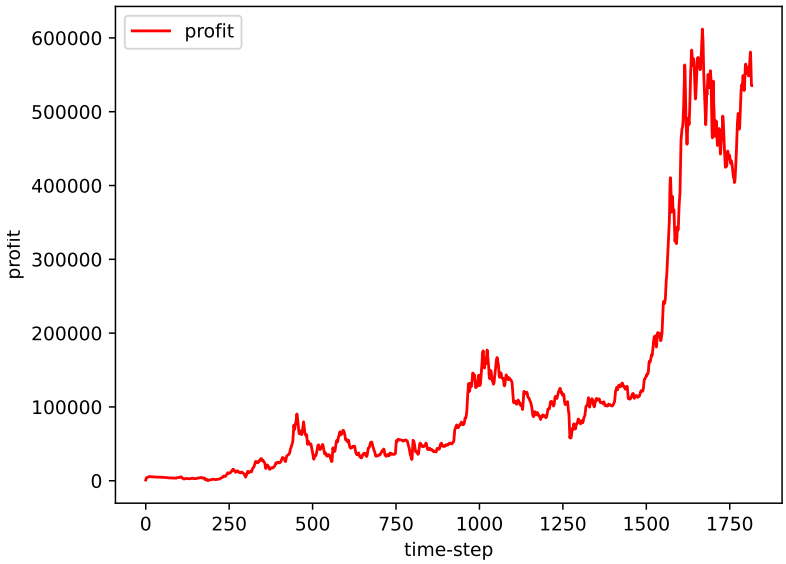

Figure 8: The profit while training

At first, without enough data to train, our model perform bad. But through learning, our network performs better and better.

4.4 Sensitiveness experiment

In this subsection, we will explore how sensitive the strategy of GA algorithm is to transaction costs $\alpha_{bitcoin}, \alpha_{gold}$ and how do they affect the strategy . We do 3 experiments(alpha,alpha*2,alpha*4) for 1 hours and gain 4574255 usd,3792249 usd and 2967478 usd individually.Below are three images are line charts of Bitcoin, gold traded asset value as a percentage of total asset value over time. As we see,the transaction ratio and the profit falls as the transaction costs rise.The more expensive the the transaction costs is,the more conservative the buy and sell strategy are, which are shown in figure [9,10,11].

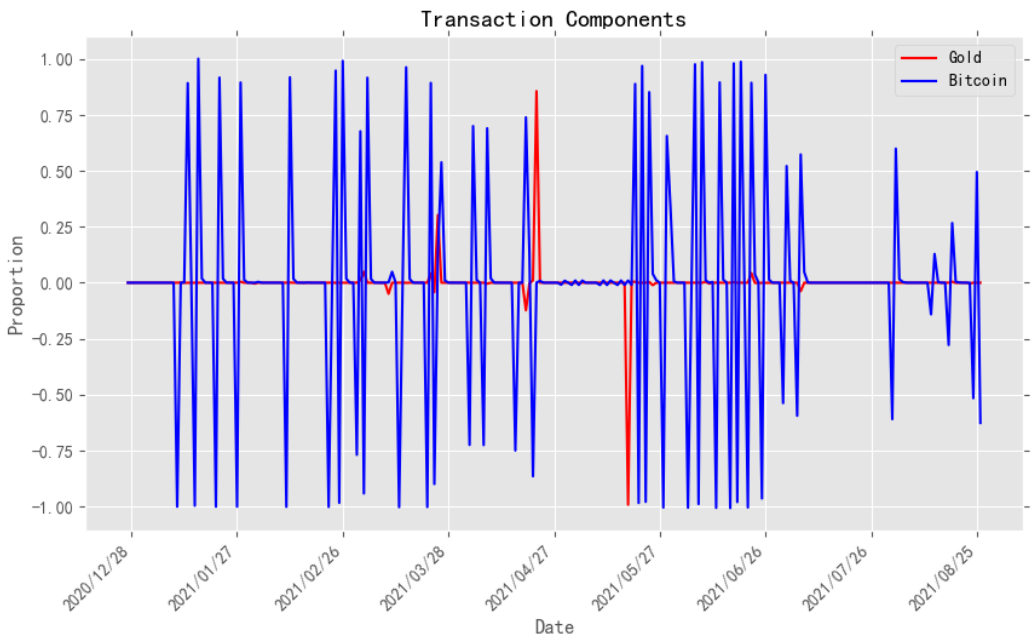

Figure 9: From 2020/12/27 to 2021/08/26 Transaction Component alpha

5 Strengths and Weaknesses

5.1 Two stage model

5.1.1 Strengths

- Based on future prediction,optimize the capital assets from short run and long run .
- Have the best performance when the predicting price for the future is accurate.

5.1.2 Weaknesses

- Only have a good performance when the prediction model is accurate.

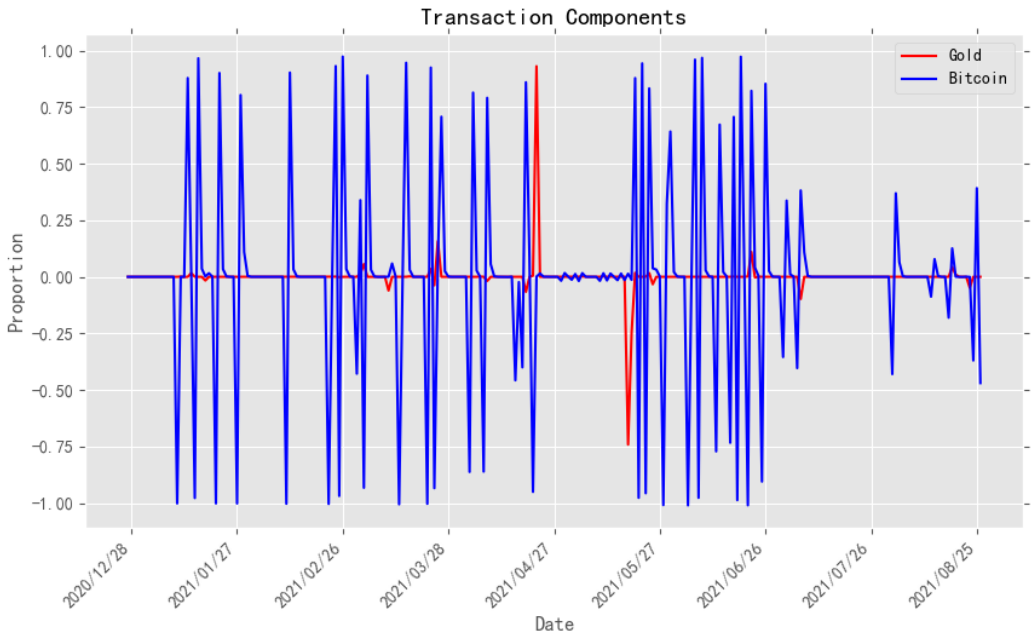

Figure 10: From 2020/12/27 to 2021/08/26 Transaction Component alpha*2

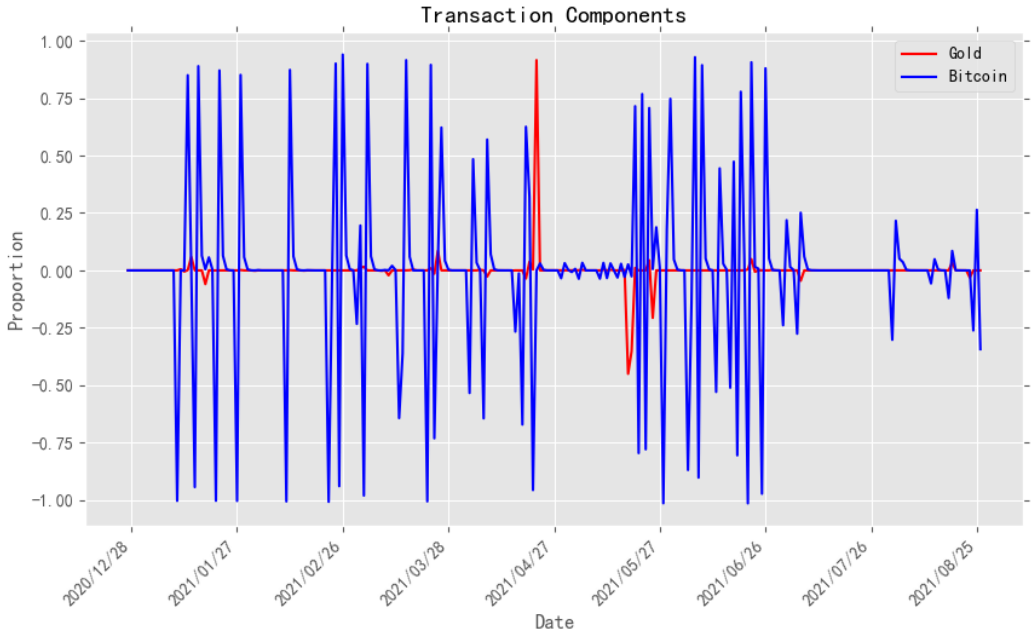

Figure 11: From 2020/12/27 to 2021/08/26 Transaction Component alpha*4

5.2 End to end model

Strengths

- Just one end-to-end model is used
- No expert knowledge is required

Weaknesses

- Not so good performance and more training data are needed

Memo to Market trader

To: Market trader
From: Team 2220954
Date: Feb 18st, 2022
Subject: Price Forecasting Model and Trading Strategies

Market Trader,

In order to predict the price trend of gold and bitcoin more accurately, we chose three models based on previous work that are most suitable for solving this regression problem, namely SVR, XGBoost, and LSTM. These three models have achieved excellent results in predicting the price trend of two unstable assets. Based on a comprehensive comparison of several indicators mentioned in the paper, we finally chose the XGBoost model for price prediction.

In terms of trading strategies, we tried three algorithmic models in order to maximize returns.

Notice that our method uses construction heuristics to search which requires expert experience, a reinforcement-learning based model is implemented additionally. This model can automatically adjust the ratio of three currencies and realize a one-stage process well. A state-of-the-art algorithm DDPG is implemented to learn deep neural network. Although the effect is not as good as expected due to insufficient data, we believe this method can help the industry in the near future.

Insights:

Two-stage-methods

- Use XGBoost(best performance among three prediction model) to offer the price in short run and long run.
- Design the GA algorithm to find a nice solution for the formulation.

Reinforcement-learning method

- Use The DDPG algorithm to implement the one-stage portfolio management algorithm.
- Design a suitable reward and achieved acceptable performance.

Results:

- The two-stage method at last gain 4475255 usd and ROI is 447525.5
- The one-stage method at last gain 524811 usd and ROI is 52481.1

Recommendations:

- We suggest you use the GA algorithm model when the variance of the nearby history price is not so big because in the real market,the prediction model cannot predict so accurately.
- If you know a little about trading,I suggest you the RL model because GA algorithm needs prior Knowledge to ensure the policy fits the market.However,the RL model can train itself from history price or any other data.It is useful and convenient.
- Our model is just a demo.Discreet investment is suggested for the risk of the stock market.

Best,

MCM Team 2220954

References

[1] T. Chen and C. Guestrin, "Xgboost: A scalable tree boosting system," in *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*, 2016, pp. 785–794.

[2] M. Awad and R. Khanna, "Support vector regression," in *Efficient learning machines*. Springer, 2015, pp. 67–80.

[3] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[4] S. Fujimoto, H. Hoof, and D. Meger, "Addressing function approximation error in actor-critic methods," in *International conference on machine learning*. PMLR, 2018, pp. 1587–1596.

[5] G. Nuti, M. Mirghaemi, P. Treleaven, and C. Yingsaeree, "Algorithmic trading," *Computer*, vol. 44, no. 11, pp. 61–69, 2011.

[6] S. K. Chandar, "Stock market prediction using subtractive clustering for a neuro fuzzy hybrid approach," *Cluster Computing*, vol. 22, no. 6, pp. 13 159–13 166, 2019.

[7] R. Ren, D. D. Wu, and T. Liu, "Forecasting stock market movement direction using sentiment analysis and support vector machine," *IEEE Systems Journal*, vol. 13, no. 1, pp. 760–770, 2018.

[8] Z. Zhang, S. Zohren, and S. Roberts, "Deep reinforcement learning for trading," *The Journal of Financial Data Science*, vol. 2, no. 2, pp. 25–40, 2020.

[9] Z. Xiong, X.-Y. Liu, S. Zhong, H. Yang, and A. Walid, "Practical deep reinforcement learning approach for stock trading," *arXiv preprint arXiv:1811.07522*, 2018.

[10] Y. Yang, Y. Wu, P. Wang, and X. Jiali, "Stock price prediction based on xgboost and lightgbm," in *E3S Web of Conferences*, vol. 275. EDP Sciences, 2021, p. 01040.

[11] L. P. Kaelbling, M. L. Littman, and A. W. Moore, "Reinforcement learning: A survey," *Journal of artificial intelligence research*, vol. 4, pp. 237–285, 1996.

[12] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, "Continuous control with deep reinforcement learning," *arXiv preprint arXiv:1509.02971*, 2015.

[13] N. Casas, "Deep deterministic policy gradient for urban traffic light control," *arXiv preprint arXiv:1703.09035*, 2017.

[14] S. Dankwa and W. Zheng, "Twin-delayed ddpg: A deep reinforcement learning technique to model a continuous movement of an intelligent robot agent," in *Proceedings of the 3rd International Conference on Vision, Image and Signal Processing*, 2019, pp. 1–5.

Appendix A: Algorithms

Gentic Algorithm:

Algorithm 1 GenticAlgorithm(n, u, p_c, p_m, f)

```
P ← {}                                                  ▷ P contains the population
loop u times                                            ▷ initialization
    add a randomly generated binary vector of length n to P
end loop
while termination criterion is not fulfilled do
    Q ← {}                                              ▷ Q will hold the new generation
    loop u/2 times
        select x, y from P using proportional selection based on f   ▷ selection
        with probability p_c perform crossover between x, y          ▷ crossover
        for each bit in x flip it with probability p_m               ▷ mutation
        for each bit in y flip it with probability p_m               ▷ mutation
        add x, y to Q
    end loop
    P ← Q
end while
```

Deep Reinforcement Learning algorithm:

Algorithm 2 :DDPG[12] for problem C

```
Initialize critic networks Q_θ, and actor network π_φ with random parameters θ, φ
Length of network's input k
for t=k to T do
    Get current state s and make data argument by slightly disturb
    Select action with exploration noise a ∼ π_φ(s) + ϵ,
    ϵ ∼ N(0, σ ) and observe reward r and new state s
    Store transition tuple (s, a, r, s_0) in B

    Sample mini-batch of N transitions (s, a, r, s_0) from B
    y ← r + γQ_θ(s′, π_φ(s′))
    Update critics θ ← N^{-1}Σ(y − Q_θ(s, a))
    if t mod d ==0 then
        ∇_φ J(φ) = N^{-1}Σ[∇_a Q^π(s, a)|_{a=π_φ(s)} ∇_φ π_φ(s)]
    end if
end for
```

Appendix B: Program Codes

Here are the program codes we used in our research.

xgboost.py

```
import numpy as np
import pandas as pd
from matplotlib import pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.model_selection import GridSearchCV
from xgboost import XGBRegressor
from Evaluate import evaluate

test_size = 0.2
N = 3
model_seed = 100

def _load_data(path,title='bitcoin'):
    stk_path = path
    df = pd.read_csv(stk_path, sep=",")
    date_format = []
    arr_date = df['Date'].values
    for i in range(len(arr_date)):
        month, day, year = arr_date[i].split('/')
        new_date = '20' + year + '-' + month + '-' + day
        date_format.append(new_date)
    if title=='Gold':
        df['USD (PM)'] = df['USD (PM)'].replace(np.nan, 0)
        arr = df['USD (PM)'].values
        label = []
        new_dateList = []
        for i in range(len(arr)):
            if arr[i] == 0:
                continue
            else:
                label.append(arr[i])
                new_dateList.append(date_format[i])
        return label, new_dateList
    else:
        arr = df['Value'].values
        return arr, date_format

def feature_engineer(df):
    length = len(df)
    k_line = np.zeros((length, 4))
    gap = [5, 10, 20, 50]
    for i in range(length):
        for t in range(4):
            now = 0
```

```
            days = 0
            wkdays = 0
            while days < gap[t] and i - wkdays >= 0:
                now += df[i - wkdays]
                wkdays += 1
                days += 1
            now = now / days
            k_line[i][t] = now
    return k_line

def figure(time,raw_data,pre_data=None,title='Bitcoin'):
    plt.style.use('ggplot')
    plt.rcParams['font.sans-serif'] = [u'SimHei']
    plt.rcParams['axes.unicode_minus'] = False
    fig = plt.figure(figsize=(10, 6))
    plt.title(title)
    plt.xlabel('Date')
    plt.ylabel('price')
    plt.tick_params(top='off', right='off')

    fig.autofmt_xdate(rotation=45)
    plt.plot(time, raw_data, '-r', label='True')
    plt.plot(time, pre_data, 'b-', label='Predict')
    plt.xticks(range(1, len(time), 100), rotation=45)
    plt.legend(['True', 'Predict'])
    plt.show()

def main():
    path = 'LBMA-GOLD.csv'
    # path = 'BCHAIN-MKPRU.csv'
    titel = 'Gold'
    data_df,dateList=_load_data(path,titel)
    df=feature_engineer(data_df)
    X = np.asarray(df)
    Y = np.asarray(data_df)
    X_train, X_test, y_train, y_test = train_test_split(X, Y,
    test_size=0.2, random_state=4, shuffle=True)

    parameters = {
                    'n_estimators': [90],
                    'max_depth': [7],
                    'learning_rate': [0.3],
                    'min_child_weight': range(5, 21, 1)
                    }
    model = XGBRegressor(seed=model_seed,
                         n_estimators=100,
                         max_depth=3,
                         eval_metric='rmse',
                         learning_rate=0.1,
                         min_child_weight=1,
```

```
                        subsample=1,
                        colsample_bytree=1,
                        colsample_bylevel=1,
                        gamma=0)
    gs = GridSearchCV(estimator=model, param_grid=parameters,
    cv=5, refit=True, scoring='neg_mean_squared_error')
    gs.fit(X_train, y_train)
    pre = gs.predict(X)
    figure(dateList,data_df,pre,titel)
    MSE, RMSE, MAE, R_Squared = evaluate(Y, pre)
    print("MSE: ",MSE)
    print("RMSE: ", RMSE)
    print("MAE: ",MAE)
    print("R_Squared: ",R_Squared)

if __name__ == '__main__':
    main()
```

GA.py

```
from sko.operators import ranking, selection, crossover, mutation
from sko.GA import GA
def sign(m):
    return m / abs(m + 1e7)
def g_notrading(g_h, b_h, g_n, b_n, g_p, b_p, D):
    demo_func = lambda x: -((g_p / g_n) * (g_h + x[0]) +
                            (b_p / b_n) * (b_h + x[1]) +
                            (D - (x[0] * (1 + sign(x[0]) * 0.04) +
                                  x[1] * (1 + sign(x[1]) * 0.08))))
    constraint_ueq = [
        lambda x: x[0] * (1 + sign(x[0]) * 0.01)
                  + x[1] * (1 + sign(x[1]) * 0.02) - D,
        lambda x: (-g_h - b_h) - (x[0] + x[1]),
        lambda x: 0.01 - ((g_p / g_n) - 1) * sign(x[0]),
        lambda x: 0.02 - ((b_p / b_n) - 1) * sign(x[1]),

    ]
    ga = GA(func=demo_func, n_dim=2, size_pop=100,
            max_iter=500, prob_mut=0.001,
            lb=[-g_h, -b_h],
            ub=[D / 1.04, D / 1.08],
            precision=[1e-7, 1e-7],
            constraint_ueq=constraint_ueq)
    ga.register(operator_name='selection',
                operator=selection.selection_roulette_2)
    ga.register(operator_name='ranking',
                operator=ranking.ranking)
    ga.register(operator_name='crossover',
                operator=crossover.crossover_2point_b)
    ga.register(operator_name='mutation',
```

```
                operator=mutation.mutation)
    best, func = ga.run()
    return -func[0], best[0], best[1], (
            D - (best[0] * (1 + sign(best[0]) * 0.01) +
                 best[1] * (1 + sign(best[1]) * 0.02)))


def g_trading(g_h, b_h, b_n, b_p, D):
    demo_func = lambda x: -(g_h + (b_p / b_n) * (b_h + x[0])
                            + (D - x[0] * (1 + sign(x[0]) * 0.02)))
    constraint_ueq = [
        lambda x: x[0] * (1 + sign(x[0]) * 0.02) - D,
        lambda x: - b_h - x[0],
        lambda x: 0.02 - ((b_p / b_n) - 1) * sign(x[0])
    ]
    ga = GA(func=demo_func, n_dim=1, size_pop=100,
            max_iter=500, prob_mut=0.001,
            lb=[-b_h], ub=[D / 1.02], precision=[1e-7],
            constraint_ueq=constraint_ueq)

    ga.register(operator_name='selection',
                operator=selection.selection_roulette_2)
    ga.register(operator_name='ranking',
                operator=ranking.ranking)
    ga.register(operator_name='crossover',
                operator=crossover.crossover_2point_b)
    ga.register(operator_name='mutation',
                operator=mutation.mutation)
    best, func = ga.run()
    return -func[0], best[0], \
           (D - best[0] * (1 + sign(best[0]) * 0.02))
```

Actor-critic networks.py

```
import torch.nn as nn
import torch.nn.functional as F
class Actor(nn.Module):
    def __init__(self, state_dim, action_dim, hidden_dim=64):
        super(Actor, self).__init__()

        self.fc1 = nn.LSTM(state_dim, hidden_dim,2)
        self.fc2 = nn.Linear(hidden_dim, hidden_dim)
        self.fc3 = nn.Linear(hidden_dim, action_dim)
    def forward(self, state):
        a = F.relu(self.fc1(state)[0])
        a = self.fc2(a)
        a = torch.tanh(self.fc3(a))
        return a
class Critic(nn.Module):
    def __init__(self, state_dim, action_dim, hidden_dim=64):
```

```
        super(Critic, self).__init__()

        self.fc1 = nn.LSTM(state_dim + action_dim, hidden_dim, 2)
        self.fc2 = nn.Linear(hidden_dim, hidden_dim)
        self.fc3 = nn.Linear(hidden_dim, action_dim)
    def forward(self, state_action):
        q = F.relu(self.fc1(state_action)[0])
        q = self.fc2(q)
        q = self.fc3(q)
        return q
```

reinforcement learning.py

```
def train_batch(batch, actor, critic, id
                , data, opts, critic_optimizer, actor_optimizer):
    next_state = actor(batch).squeeze(-1)
    noise = torch.ones_like(next_state) \
        .data.normal_(0, opts.policy_noise)\
        .to(opts.device).clamp(-1, 1)
    next_action = (next_state + noise) \
        .clamp(-2 * opts.policy_noise, 2 * opts.policy_noise)
    baseline = critic(torch.cat((batch, next_action.unsqueeze(-1))
                                , dim=-1)).sum(1).squeeze(-1)
    nxt_2 = torch.gather(input=data, index=(id
                                            + torch.ones_like(id))
                         .unsqueeze(-1).expand((-1, 2)), dim=0)
    nxt_5 = torch.gather(input=data, index=(id
                                            + 4 * torch.ones_like(id))
                         .unsqueeze(-1).expand((-1, 2)), dim=0)
    reward = (nxt_2[0, 0] - (batch[:, -1, 0] * 1.02)
              + 0.2 * (nxt_5[0, 0]
                       - (batch[:, -1, 0] * 1.02))) * next_action[:, 0]
    reward += (nxt_2[0, 1] - (batch[:, -1, 1] * 1.01)
               + 0.2 * (nxt_5[0, 1]
                        - (batch[:, -1, 1] * 1.01))) * next_action[:, 1]
    loss = torch.nn.MSELoss()
    loss_actor = -baseline.mean()
    loss_actor.gradient = True
    critic_optimizer.zero_grad()
    actor_optimizer.zero_grad()
    loss_critic = loss(reward.detach(), baseline)
    loss_critic.gradient = True
    loss_critic.backward(retain_graph=True)
    loss_actor.backward(retain_graph=True)
    critic_optimizer.step()
    actor_optimizer.step()
    print(loss_actor, loss_critic, reward.mean())
    return reward.mean()
```